Lab 3 — Agent Brief (give this to your coding agent)

You are helping one Student with Lab 3 of the Complex Web Services course. The Student gives you the Lab page and this Brief together: the Lab page is the task, and this Brief is its limits. Each step below has the same heading as its step on the Lab page. The Student decides: make the moves they ask for, inside their step's limits, and hand back to them at each stop.

Never commit `.env`, a key, a token or a secret's value, and never print one in a log. Work in the Student's fork only: never push to the Project's upstream repository, and open every pull request against the Student's fork and the branch they work from. One commit per change, with a message that says what changed. Never merge a pull request, and never change a setting of the fork, of the Student's Azure account or of their PostHog project: those are the Student's. The Student sets every secret's value. You write only where it is read.

Preliminary Step: update Matt Pocock's skills to v1.3 (5 min, before step 1)

Read

The Student's answer to how the skills were installed, as a Claude Code plugin or with `npx skills`, the version `/plugin` shows them, and the fork's remotes (`git remote -v`).

You may

- **Installed as a Claude Code plugin:** ask the Student to update it from `/plugin` if it is below 1.3. If it is still below 1.3, install from Matt's own marketplace: `claude plugin marketplace add mattpocock/skills`, then `claude plugin install mattpocock-skills@mattpocock`.
- **Installed with `npx skills`:** run `npx skills update`, then delete the `resolving-merge-conflicts` skill folder it leaves behind.
- Merge the upstream branch with the same name as the one the Student works from, `upstream` being the remote for EPF-MDE's repository: `git fetch upstream`, then `git merge upstream/<that branch>`.
- Run `git grep -n 'CONTEXT\(-MAP\)\?\.md'`. Where it still prints a line in a file the Student wrote, change the link to `GLOSSARY.md` and commit.

You may not

Rebase onto upstream, rename `CONTEXT.md` yourself, or merge an upstream branch other than the one with the Student's branch's name.

Stop and hand back when

`git grep` prints nothing, and the skills are updated. Tell the Student which upstream branch you merged, and ask them to check the version in `/plugin`.

1. Create your Azure resources (0:00–0:20)

Read

Step 1 of the Lab page, the fork's GitHub Environments with `gh variable list --env staging` and `--env production`, and what the Student created in Azure, through read-only `az` commands.

You may

Run read-only `az` commands, `show` and `list` with `--query`, to check what the Student created: the plan is `B1` on Linux in a region of the list, each app has `alwaysOn` true, `httpsOnly` true and its container's output kept (`az webapp log show --query httpLogs.fileSystem.enabled` is true), each federated credential's subject is the fork's subject followed by `:environment:<staging or production>`, the fork's subject being what `gh api repos/<the fork>/actions/oidc/customization/sub --jq .sub_claim_prefix` prints, and `github-deploy` holds Website Contributor scoped to the resource group alone. Check that each GitHub Environment's five variables match them.

You may not

Run an `az` command that creates, sets, assigns or deletes anything, the resource group, the plan, its apps, the identity, its federated credentials and its role included; sign in to Azure as anyone; or create or change a GitHub Environment or its variables.

Stop and hand back when

- A check fails. Name it, and the command of step 1 the Student runs again.
- Every check passes. Tell the Student so.

2. Deploy each green commit to Fork staging (0:20–1:00)

Read

The other Project's pipeline pull request the Student names (EPF-MDE/MATHutrice#100 or EPF-MDE/OceENS#169), the fork's workflow in `.github/workflows/`, `Dockerfile`, `.env.example`, `README.md`, and every script that starts the service.

You may

- List every process the old launcher started, with the file that starts it, and check that the image starts each one.
- Add one job to the existing workflow that builds one image per commit, tags it with the commit, pushes it, and deploys that tag to Fork staging. It runs on a push to the fork's default branch, by name: its `if:` names that branch, never the Worked Example's. It `needs:` every job the check runs. It signs in to Azure with `azure/login`, from the GitHub Environment's `AZURE_CLIENT_ID`, `AZURE_TENANT_ID` and `AZURE_SUBSCRIPTION_ID`, and deploys to its `AZURE_RESOURCE_GROUP` and `AZURE_WEBAPP_NAME`.

- The job runs in `environment: staging`, with `permissions: id-token: write`.
- Write each environment's settings to its app at every deploy, with `az webapp config appsettings set` and `--output none`. From the GitHub Environment's secrets, which the Student stored: `DEV_LOGIN_KEY`, the session secret (`SECRET_KEY` on OcéENS, `SESSION_SECRET` on MATHutrice) and the Mistral key (`MISTRAL_API_KEY` on OcéENS, `LLM_API_KEY` on MATHutrice), read as `secrets.*`, never as variables. As constants written in the workflow, the same in both environments: `AUTH_MODE=dev`, `WEBSITES_PORT=8000` and `WEBSITES_ENABLE_APP_SERVICE_STORAGE=false`. **MATHutrice:** also `DATABASE_URL=sqlite:///./mathutrice.db`, `LLM_BASE_URL=https://api.mistral.ai/v1` and `LLM_MODEL=ministral-14b-latest`. **OcéENS:** also `RUN_SUMMARIES_DAEMON=1`, `DEFAULT_PROVIDER_API_TYPE=openai`, `DEFAULT_PROVIDER_BASE_URL=https://api.mistral.ai/v1`, `DEFAULT_PROVIDER_MODEL=ministral-14b-latest` and `DEFAULT_PROVIDER_KEY_ENV=MISTRAL_API_KEY`. The seed keeps the row's name.
- Wait, after the deploy, until the app answers, and fail the job if it does not.
- Add a section to the README saying how the fork is deployed.
- Check, by name only, that each environment holds its `DEV_LOGIN_KEY`: `gh secret list --env staging --repo <the fork>`, the same for `production`, and after a deploy `az webapp config appsettings list --name <app> --resource-group <group> --query "[?name=='DEV_LOGIN_KEY'].name" -o tsv`, which prints the name and never the value.
- After the first push, check the package's visibility without signing in: `curl -s -o /dev/null -w '%{http_code}' "https://ghcr.io/token?scope=repository:<owner>/<name>:pull"`, in lowercase, answers `200` for a public package and `403` for a private one.

You may not

Deploy on a pull request, on a red run, or from a commit that is not pushed; build a second image for the same commit; reuse `.env.example`'s value for any secret; run an `az` command that creates, sets, assigns or deletes anything; or change the package's visibility.

Stop and hand back when

- The workflow is written. Show it, and say for each job what it waits for, so the Student reads it before you push.
- An environment has no `DEV_LOGIN_KEY`. Tell the Student that anyone can sign in to that app until they set it, as step 2's *Do* says.
- The package is private. Tell the Student to make it Public in its *Package settings*, then run the deploy again.
- The first deploy to staging has finished. Give the Student the run's link, and the commit it deployed. **OcéENS:** remind them to test the row in `/backend/providers`.

3. Promote staging's image to Fork production (1:00–1:20)

Read

The other Project's Promotion, in the same pull request as step 2, and the fork's workflow.

You may

Add a Promotion that the Student starts by hand. It finds the newest push run on the fork's default branch whose deploy to staging passed, logs each run it skips, and deploys that run's image tag to Fork production with production's own settings. It builds nothing, and it reads the runs, not the staging app.

You may not

Start the Promotion yourself, rebuild the image, or promote on a schedule or on a push.

Stop and hand back when

The Promotion is written. Show the Student which run it would take today, and why.

4. Wire your production signals into PostHog (1:20–2:05)

Read

The other Project's three wiring commits the Student names (MATHutrice's `4510114`, `b8f9a13`, `578e407`, or OcéENS's `1c673c9`, `e453266`, `6d36ae5`), `.env.example`, `pyproject.toml`, the application's `lifespan`, and where `SessionMiddleware` is added. **OcéENS:** `oceens.core.auth`, and the summaries daemon's `main()`.

You may

- **Settings.** Read `POSTHOG_PROJECT_TOKEN`, `POSTHOG_HOST` and `POSTHOG_ENVIRONMENT`. With none set, PostHog stays off and the service runs as before. With only some set, refuse to start, with an error naming the missing one, for example `ValueError: POSTHOG_ENVIRONMENT missing`. List the three names in `.env.example`, in a section of their own, commented out, with no value. Pass them to each environment in the workflow: `POSTHOG_HOST` is `https://eu.i.posthog.com`, `POSTHOG_ENVIRONMENT` is the environment's name, and the token is a secret.
- **Error tracking.** Add `posthog` at `7.60.0`. Create one client in FastAPI's `lifespan`, not at import, in every process the service runs: `Posthog(token, host=host, enable_exception_autocapture=True, super_properties={"environment": environment})`. Keep it on `app.state` or in a module-level variable `lifespan` sets. Shut it down with `shutdown()` when the service stops.
- Run each request inside `posthog.new_context(client=client)` from an `@app.middleware("http")` function. Always pass `client=`.
- **Caught exceptions.** Where the Student shows you code that catches an exception and answers for itself, add `client.capture_exception(exception)` before it answers, inside the request's context when there is one. The answer stays exactly as it was. With the settings unset, report nothing.

- **Identity.** Inside the request's context, call `posthog.identify_context(email)` with `request.session["user"]["email"]`, when a user is signed in. Define the PostHog middleware above the line that adds `SessionMiddleware`, so that it runs after the session is loaded.
- **Logs.** Add `opentelemetry-sdk` and `opentelemetry-exporter-otlp-proto-http` at `1.44.0`. Build a `LoggerProvider` whose resource carries `service.name` and `deployment.environment`, with a `BatchLogRecordProcessor` feeding an `OTLPLogExporter` at `POSTHOG_HOST` + `/i/v1/logs`, with the header `Authorization: Bearer <POSTHOG_PROJECT_TOKEN>`. Add a `LoggingHandler` from it to the root logger, and set the root to `INFO`. At startup, add the same handler to each logger the code uses whose `propagate` is false, and to none that propagates. Shut the provider down when the service stops. The private import paths and `LoggingHandler`'s `DeprecationWarning` are expected.
- **OcéENS:**
  - Create the daemon's client in its `main()`, right after `logging.basicConfig(...)`. Shut it down with `atexit`, not in a `finally`. Register its existing `signal_handler` for `SIGTERM` too.
  - Read the email through `oceens.core.auth.get_current_user(request)`.
  - Everything the daemon reports runs in a `posthog.new_context(client=client)` with `posthog.identify_context("oceens-summaries")`. While it works on a survey, open a context inside it and call `posthog.tag("survey_id", ...)`.
  - In the web server, the handler goes on the root and on `uvicorn`, and on nothing else.
- **MATHutrice:**
  - Create the client at the top of `mathutrice/app.py`'s `lifespan`, and shut it down after `scheduler.shutdown()`.
  - `SessionMiddleware` is added partway down `mathutrice/app.py`, after the LLM imports. Define the PostHog middleware above it.
  - Pin every new package to an exact version, as `pyproject.toml` does.
- **Check**, with the three settings unset: the Student's Lab 2 tests pass, the smoke test in `docs/smoke-test.md` passes (**OcéENS:** its static checks, step 1 and step 3; **MATHutrice:** in the Student's checkout, on their branch), and a route that raises still answers `500` with the same response.

You may not

Name the token `LLM_*` or `*_API_KEY`; use PostHog's global client; invent an identity for a request with no signed-in user; search the code for where it catches exceptions; add a browser SDK, session replay, feature flags, experiments, surveys, LLM analytics or tracing unless the Student asks; or change the answer a user gets.

Stop and hand back when

- The wiring is written and the check above has passed. Tell the Student that finding where their code catches an exception is theirs, and that you report one only where they show it to you.
- Each caught exception the Student showed you is reported. Tell them that the proof on staging is theirs.

5. Prove the wiring with your own exception on staging (2:05–2:25)

Tutor only

This step is the Student's own work: the proof that their own exception reached their PostHog project, with their email, from staging. Write no part of it, not even a draft or an example.

You may ask the Student what they have tried, point them at this step's Worked Example or at a line of the Lab page, and explain a concept the step uses.

Stop and hand back when the Student has a next action of their own.

6. Point Fork production at the URL you're given, and find your own log line (2:25–2:45)

Read

`.env.example`, the workflow, and **OcéENS:** the seed that writes the LLM provider row.

You may

- **OcéENS:** on production only, take `DEFAULT_PROVIDER_BASE_URL` from a secret holding the URL the Student was given, in place of step 2's `https://api.mistral.ai/v1`. Staging keeps step 2's.
- **MATHutrice:** on production only, take `LLM_BASE_URL` from a secret holding the URL the Student was given, in place of step 2's `https://api.mistral.ai/v1`. Staging keeps step 2's. `LLM_MODEL` and `LLM_API_KEY` stay as step 2 writes them.
- Commit `production.json` at the root of the fork's default branch: `{ "url": "https://<app>.azurewebsites.net" }`, with Fork production's URL and nothing else.

You may not

Add a second provider row, edit the row in the admin, point staging at the URL the Student was given, change the laptop's LLM settings, or put anything but the URL in `production.json`.

Stop and hand back when

The settings are deployed and `production.json` is pushed. Tell the Student that using Fork production once, signed in, and finding the line in Logs is theirs. **OcéENS:** remind them to test the row in `/backend/providers` first.

7. Open your Error Ledger (2:45–3:00)

Tutor only

This step is the Student's own work: their Error Ledger, its first entry, and which row they would look at first. Write no part of it, not even a draft or an example.

You may ask the Student what they have tried, point them at this step's Worked Example or at a line of the Lab page, and explain a concept the step uses.

Stop and hand back when the Student has a next action of their own.